

长颈鹿在 LLM 中扩展上下文长度的历险记

Arka Pal,* Deep Karkhanis, Manley Roberts, Samuel Dooley,
Arvind Sundararajan, Siddhartha Naidu

算盘人工智能

摘要

依赖于注意力机制的现代大型语言模型 (LLM) 通常是在固定的上下文长度下进行训练的，这就对它们在评估时所能处理的输入序列的长度规定了上限。为了将这些模型用于比训练时上下文长度更长的序列，人们可能会采用不断增加的上下文长度外推方法系列中的一些技术--其中大多数技术都侧重于修改注意力机制中使用的位置编码系统，以指示输入序列中的标记或激活位置。我们对现有的基于 LLaMA 或 LLaMA 2 模型的上下文长度外推方法进行了广泛的调查，并引入了一些我们自己的设计--特别是一种新的截断策略，用于修改位置编码的基础。

我们使用三个新的评估任务 (FreeFormQA、AlteredNumericQA 和 LongChat-Lines) 以及困惑度对这些方法进行了测试。我们在 HuggingFace 上公开发布了这三个任务的数据集。我们发现线性缩放是扩展上下文长度的最佳方法，并表明在评估时使用更长的缩放可以获得进一步的收益。我们还发现在截断基础上的外推能力很有前途。为了支持这一领域的进一步研究，我们发布了三个新的 13B 参数长语境模型，我们称之为 **Giraffe**：4k 和 16k 上下文模型由基础 LLaMA-13B 训练而成，32k 上下文模型由基础 LLaMA2-13B 训练而成。我们还发布了复制我们结果的代码¹。

1 简介

近年来，变换器[1]凭借其灵活性和在超大数据集上训练的便利性[4, 2, 5]，已成为各种自然语言建模任务[3]中的主流神经网络架构。因此，这些神经网络被称为“大型语言模型”(LLMs) --“大型”既指训练数据集的大小，也指其参数数量（以及相关的训练和环境成本）。

标准变换器架构的一个关键因素是它本身对输入元素的排序不敏感。注意自然是一种类似集合的操作，其中元素的位置并不重要[1]。然而，元素的顺序对许多重要任务至关重要，如解析自然语言、编码、预测等。因此，有必要在 LLM 的输入中注入位置信息，通常采用位置编码的形式。

位置编码方案的一个必要条件是上下文长度外推法：使用 LLM 进行推理的输入长度要长于它所训练的输入长度。由于变换器中注意力机制的复杂度呈二次方增长，因此通常不可能在较大的上下文长度上进行训练。增加上下文长度的好处是多方面的--可以阅读更长的文档和论文、

*arka@abacus.ai。

¹Github repo: <https://github.com/abacusai/Long-Context>。

在由 LLM 驱动的聊天机器人中与用户进行长时间对话、在更大的代码库中工作，等等，这些都能提高内部一致性。我们可以将上下文长度外推分为两种主要模式。首先是 *微调外推法*，即允许先前在较短上下文中预设的模型根据较长上下文长度进行微调或更新模型权重。此外，还有 *"零点外推法"*，即先前在较短上下文中预处理过的模型立即在较长上下文中进行评估，权重与较短上下文模型相同。

在本文中，我们主要关注 *零点外推法*，并做出了以下 **主要贡献**：

不同语境外推方案的基准 我们对使用预训练基础模型进行语境长度外推的方法进行了调查，并尝试了一些我们自己的发明。特别是，我们提出了一种新的位置编码 *截断* 基础。本文对预训练模型的关注也不同于其他文献[6, 7]中的研究，后者倾向于使用选定的位置编码方案从头开始训练。如上所述，尽管 LLM 取得了成功，但训练 LLM 是一项成本高昂的工作。著名的闭源模型包括 GPT-4 [8] 和 Claude [9]。最近，Meta AI 团队发布了开源 LLaMA [10]，随后又发布了改进版 LLaMA2 [11]。我们认为，训练这种性质的有竞争力的基础模型所需的资源仍将局限于少数大型公司。因此，必须能够根据最终用户的需要修改模型--理想情况下，只需使用一小部分计算能力。

我们的主要发现如下

- 线性插值是最好的上下文长度外推方法。
- 所有上下文长度外推法都会降低任务的准确性，即使它们能提供一致的输出长度（并且易解性得分仍然合理）。
- 通过在评估时使用比微调时更高的比例因子，可以进一步增加上下文长度，但似乎只能增加到 2 倍。

公开发布 LLM 权重和评估数据集 我们发布了两个新的 13B 模型的权重，这两个模型是在 HuggingFace 上用扩展上下文长度为 $16k^2$ 和上下文长度为 $4k^3$ 的基础 LLaMA 训练出来的。我们还发布了一个根据基础 LLaMA 2 训练的长度为 32k 的 13B 模型⁴。我们称这个模型系列为 *"长颈鹿"*。此外，我们还发布了三个数据集（LongChat-Lines⁵、FreeFormQA⁶和 AlteredNumericQA⁷），以评估这些模型和其他模型的长语境性能。LongChat-Lines 是一项键值细粒度检索任务。FreeFormQA 和 AlteredQA 是基于自然问题数据集 [12] 的。问题解答数据集现有的一些研究 [6, 7] 仅将文档语料库评估集上的易错度作为外推性能的衡量标准。我们发现，在衡量长语境性能时，易错性得分不如我们引入的任务灵敏。

2 相关工作

RoPE 在这项工作中，我们检验了 LLaMA [10] 所选择的位置编码在上下文长度超过基础模型训练长度时的有效性。LLaMA 使用的位置编码是 RoPE（旋转位置嵌入）[13]。RoPE 的工作原理是以不同的速度旋转查询矩阵和关键投影矩阵的切片。因此，举例来说，即使查询和关键字投影到相同的编码，它们也会根据在序列中的位置以不同的速度旋转。如果它们随后未对齐，它们的点积就会小于它们未旋转时的点积。反之，它们

² <https://huggingface.co/abacusai/Giraffe-v1-delta-13b-scaled-16> ³

<https://huggingface.co/abacusai/Giraffe-v1-delta-13b-scaled-4> ⁴

<https://huggingface.co/abacusai/Giraffe-v2-13b-32k> ⁵

<https://huggingface.co/datasets/abacusai/LongChat-Lines>⁶

https://huggingface.co/datasets/abacusai/WikiQA-Free_Form_QA

⁷https://huggingface.co/datasets/abacusai/WikiQA-Altered_Numeric_QA

可能会变得更加一致，从而导致更大的点积和注意力分数。在 RoPE 中，这种旋转在查询的所有 2 片上以不同的速度进行，并在嵌入维度中进行关键旋转，从而使模型能够建立一个复杂的注意力分数距离函数。使用 RoPE 方法的主要优点之一是，它能从数学上确保注意力分数函数只取决于查询和关键字之间的相对距离，而不是它们的绝对位置。这被认为是 LLM 的理想特性[13, 14]。

ALiBi 虽然 RoPE 成功实现了这一目标，但 ALiBi [6] 上的工作表明，RoPE 无法执行零镜头上下文长度外推。ALiBi 论文表明，当测试的上下文长度超过模型在训练期间所见的长度时，RoPE 的性能很快就会下降；论文还介绍了自己提出的替代方案，该方案在基准测试中显示出更出色的外推能力。不过，ALiBi 也有自己的不足之处：它使用简单的线性函数来调节注意力分数的距离，这意味着它不能像 RoPE 的傅立叶基础那样表示复杂的距离-注意力函数。此外，ALiBi 对每个头部只使用一个这样的函数，进一步降低了表达能力。这也许可以解释为什么，尽管 ALiBi 的确能进行推断，但在 MMLU [2] 和 LMSys 竞技场等衡量人类偏好的基准测试中，使用 ALiBi 的模型的性能却不如基于 RoPE 的模型[15]。

xPos Sun 等人[7]研究了 RoPE 无法成功推断的原因，并确定这是由于高频成分的影响，即使标记之间相距甚远，也会在注意力分数中产生残余噪声。他们试图通过在 RoPE 中添加指数衰减振幅项来解决这一问题。这种被称为 xPos 的新方法可以比低频分量更快地衰减这些噪声高频分量。这种方法在从头开始训练 LLM 方面取得了很好的效果[7]，其直观性与我们对 RoPE 不足之处的假设相吻合。然而，Sun 等人并没有在我们感兴趣的环境中进行实验：使用 RoPE 对模型进行预训练，并观察是否可以（通过有限的微调）哄骗它学习 xPos 编码。此外，他们的实验表明，要实现外推法，顺时针因果注意（Blockwise Causal Attention）是必不可少的。

线性缩放/位置内插法 kaikendev [16] 和 Meta [17]。团队同时报道了这种简单而有效的上下文长度外插技术这里使用的方法是简单地将位置矢量除以一个缩放因子，该因子适合原始模型上下文长度内的输入。这种技术的直觉是利用 LLM 的插值能力，而不是依赖外推法。一个众所周知的现象是，神经网络倾向于在先前所见值的范围内进行内插，而不是在该范围外进行外推（例如 [18]）。在位置编码的特殊情况下，[17] 称位置插值可以避免与外推相关的注意力值大量数值爆炸的风险。我们对这一方案及其变体进行了多次实验，并在本文中报告了实验结果。

随机位置编码 Ruoss 等人在 [19]。中介绍了这种方法在训练过程中，他们从 $[1, L]$ 范围内均匀无替换地抽取 N 个样本，随机生成位置向量，其中 N 是训练上下文长度， L 是一个大值，大于（假定事先已知的）最大评估上下文长度。然后，这些采样位置按递增顺序排列，作为模型在评估时看到的位置输入。在评估过程中，位置输入 $[1, \dots, M]$ 会提供给模型。作者声称，这种方法提高了上下文长度外推的性能。我们独立得出了与本文类似的想法，但随机化的方法是从近似于 $[1, N]$ 范围内的子整数位置进行抽取；更多详情请参见第 4 节。我们还注意到，Ruoss 等人研究了如何利用这种方案从头开始训练 LLM，而我们主要感兴趣的是利用随机化对预训练的 LLM 进行事后微调，以实现上下文长度外推。

3 评估长背景外推法

本文提出的主要问题是扩展 LLM 的上下文长度容量。为了评估这一点，文献中常用的指标是 *plexity* [6, 7, 13]。然而，正如我们在第 5.3 节中所展示的，在评估模型使用较长上下文窗口的能力时，*perplexity* 的粒度略显粗糙。我们的直觉是，在许多自然语言数据集中，即使模型只关注上下文窗口有限范围内的信息（例如最后 512 个词组），也能获得合理的复杂度分数。例如，位置编码方案只需屏蔽掉上下文中长度大于模型训练长度的任何元素（以及注意头中的内部关键字和查询激活），就能获得合理的困惑度得分，但在我们下面描述的任务中表现不佳。

在现有工作的基础上，我们进一步研究了模型在遇到可验证答案的问题时的 *准确性*。通过使用这一指标，我们可以评估模型是如何利用额外的上下文信息来响应提示的。我们依靠两类评估任务来评估模型从长输入上下文中提取和使用信息的能力：一类是键值检索任务，另一类是问题解答任务。通过使用这两类任务，我们强制要求模型关注完整的上下文，以获得高准确率。我们认为，检索任务是一种更纯粹的信息检索测试，不存在许多自然语言偏差。然而，检索任务是一个有些人造的构造，LLM 在训练过程中很可能不会看到它，因此我们还加入了问题解答，以复制更真实的任务。

LongChat-Lines 我们从合成的细粒度键值检索任务开始，该任务最早在 [20] 中提出，[21]。也曾使用过虽然这些作品在 LLM 的标准上下文中表现出色，但它们缺乏评估我们的实验所需的较长的上下文长度。因此，我们使用了与 [20, 21] 相同的任务，但生成了更长上下文长度的额外样本。这项任务会给模型一个提示，其形式如下

- line grotesque classmate: REGISTER CONTENT is< 42527 >
- 行 不完美的公牛: 注册内容为< 3119 >
- 行 supreme-inversion: REGISTER CONTENT is< 13960 >
- ...

我们要求模型记住每一行 REGISTER CONTENT 对应的值，并在最后要求模型检索特定行的值。通过改变提示的行数，我们可以控制上下文的长度。我们发布了比 [20] 更长的任务版本，同时还发布了该任务的生成脚本。

WikiQA 我们还从自然问题 [12] 数据集中创建了两个新数据集，专门用于较长语境的评估，我们统称为 WikiQA。在这项评估中，给 LLM 的提示格式是维基百科文档，然后是一个与该文档相关的问题；要求模型回答该问题。我们确保问题的答案是一个简短的答案，要么是一个单词，要么是一个小句子，并且在给 LLM 作为输入的文档中具有完全匹配的字符串。我们称这项任务为 **自由格式 QA (FFQA)**。

然而，基于维基百科的数据集可能存在的一个问题是，模型可能会根据其预训练的语料库做出正确的回答，而不是专门使用上下文中的信息。为了解决这个问题，我们创建了另一个“改变的”数据集，我们称之为 **“改变的数字 QA” (AltQA)**。这个数据集只包含有数字答案的问题。在这里，我们将答案以及答案在文档中的每次出现都改为不同的数字，从而确保 LLM 必须关注上下文，而且只能关注上下文，才能给出正确答案。修改过程如下：

自由形式 WikiQA	更改后的数字 WikiQA
文档 》是但丁《神曲》中继《地狱篇》和《炼狱篇》之后的第三部，也是最后一部。这是一个寓言故事，讲述了但丁在象征神学的贝阿特丽斯的引导下在天堂的旅程。在诗中，天堂被描绘成一系列环绕地球的同心球体，由月亮、水星、金星、太阳、火星、木星、土星、恒星、Primum Mobile 和最后的 Empyrean 组成。它写于 14 世纪初... 问题 谁是但丁在天堂的向导 参考答案: 比阿特丽斯	文档 ...希腊曾两 次 举办夏季奥运会，一次是 1896 年的首届现代奥运会，另一次是 2004 年 2009 年的奥运会。这两届奥运会都在雅典举行，雅典与巴黎和洛杉矶是两次举办奥运会的城市，伦敦是唯一三次举办奥运会的 城市。希腊首都雅典还举办过 1906 年的校际运动会。 问题 希腊上一次举办奥运会是什么时候？ 参考答案: 2009* *而不是 2004 年

图 1：来自 WikiQA 数据集的 QA 示例片段。

- 如果答案是年份，而年份的出现频率很高（即在 1000-2100 之间），我们就会将其改为一个不同的随机值，其范围在原始值的 +/- 10 之间。我们将年份作为一种特例处理，避免因日期值非常不合时宜而破坏文件的整体连贯性。
- 如果答案是任何其他数字，我们会将其改为一个具有相同位数的随机数字。

图 1 重点展示了维基问答数据集中的示例。由于我们应用中的上下文较长，因此答案在上下文中的位置可能会对模型回答问题的能力产生重要影响。因此，我们利用 WikiQA 的两个任务来分析 LLM 的性能，即答案位置在文档中的移动情况（在开始的 10%、最后的 10%，或随机移动到其他任何位置），以及问题在提示开始或结束时的移动情况，以复制 [22] 的分析结果

4 上下文长度外推技术

我们研究了几种上下文长度外推技术，包括现有的方法（或对它们的细微变化）以及我们自己新提出的方法。

4.1 现有的上下文长度外推技术

有几种方法可以将 RoPE 位置编码适应于更长的上下文长度。我们对以下技术进行了评估。

线性缩放/位置插值 在这里，位置向量被一个缩放因子除以。因此，如果原始模型是在 $[0, 1, \dots, 2048]$ 的位置范围内训练的，那么新模型将看到 $[\frac{0}{x}, \frac{1}{x}, \dots, \frac{2048}{x}]$ ，其中 x 是缩放因子。

xPos 我们想研究用基础模型的 RoPE 编码方案训练的检查点是否可以微调为 xPos [7] 方案。除了对整个注意力模块进行修补以处理 xPos 对键和查询的独特转换这一编程障碍外，这种适应性带来的主要问题是 xPos 对浮点精度的敏感性。这种方法依赖于用具有较大（绝对）指数的数值来缩放键值；这些数值随后会在与查询的点乘中抵消。然而，对于较长的上下文，大数值实际上会超出 float16 支持的范围。我们选择

为了解决这个问题，我们在 float32 中执行了核心注意力操作，但代价是训练速度降低了 2 倍。

随机化位置编码 在这里，我们随机化位置值之间的距离，在 $0 < \epsilon \ll 1$ 的范围 $[\epsilon, 2]$ 内，而不是使用典型的 $[0, 1, \dots, n]$ ，因为 $[0, 1, \dots, n]$ 具有大小为 1 的固定间隔。这种方法背后的直觉是，通过在微调时向模型展示许多不同的位置内距离，模型将能够在评估时泛化到任何细粒度位置的选择，从而通过选择更小的分割有效地增加上下文长度。这与 Ruoss 等人[19]中描述的程序有些相似。我们将上限设定为 2，这样模型就能预期看到 n 的最终位置（如 $E[X] \approx 1$ for $X \sim U(\epsilon, 2)$ ）。我们还设定了 ϵ 的正值、非零下限，以避免因数值精度有限而出现位置混叠问题。

4.2 新提出的上下文长度外推技术

功率缩放 在最初的 RoPE 中，使用的基数为

$$\Theta = \{\vartheta_i = 10000^{\frac{2(i-1)}{d}} \mid i \in \{1, 2, \dots, \frac{d}{2}\}\} \quad (1)$$

其中 d 是嵌入维数。我们使用以下公式给出的基础

$$\Theta^* = \{\vartheta_i^* = \vartheta_i \left(1 - \frac{2i^k}{d}\right) \mid i \in \{1, 2, \dots, \frac{d}{2}\}\} \quad (2)$$

其中 k 为待设置参数。通过这种变换，基础的高频（短距离）元素受到的影响要小于低频（长距离）元素，后者的频率更低--见图 2。这样做的目的是希望模型在列车运行过程中，在没有看到周期函数全部范围的情况下，可以对低频进行复杂度较低的外推，从而更好地进行外推。然而，一个潜在的问题是，模型依赖于不同频率之间的特定关系，而线性变换保留了这些关系，但非线性变换却破坏了这些关系。

截断基础 从等式 1，开始我们改用应用给出的基础：

$$\vartheta_i^* = \begin{cases} \vartheta_i & \text{为 } \vartheta_i \geq b, \\ \rho & \text{为 } < \vartheta_i < b, \\ 0 & \text{为 } \vartheta_i^* \leq a. \end{cases} \quad (3)$$

其中， ρ 是一个相对较小的固定值，而 a 和 b 则是选定的截止值。这里的想法是，我们希望保留基础的高频成分，但将低频元素设置为一个恒定值--本例中为 0。通过明智地选择截止值 a ，模型将在微调过程中体验过所用上下文长度中基础的所有值（由于正弦和余弦函数的周期性），因此应能更好地推断出更大的上下文长度，以便进行评估。不过，模型仍然需要能够区分它所训练的整个上下文之间的距离，因此我们也加入了 ρ 固定频率。总之，我们希望有了这个基础，模型就可以避免在整个 RoPE 基础上学习复杂系数的问题，而是在较长的距离上学习平滑函数（如论文 [17]）。中所示

在图 2，中我们直观地比较了标准 RoPE 基、功率缩放和截断所产生的频率。

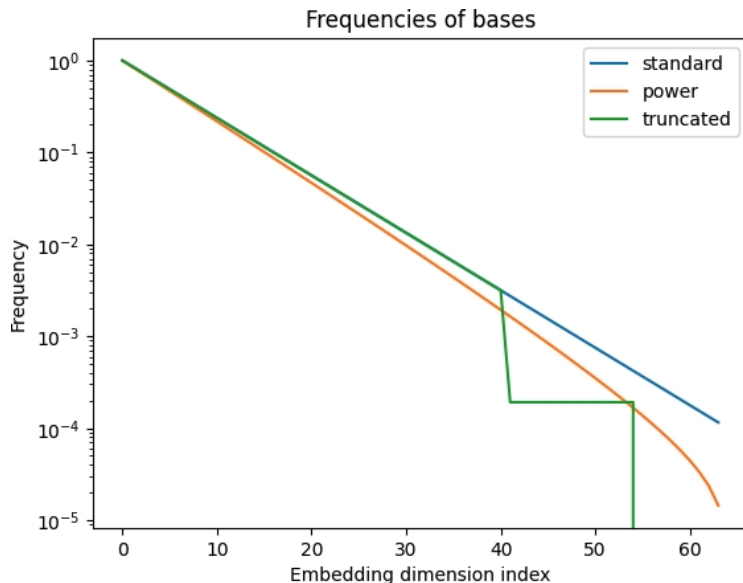


图 2：标准 RoPE 基础与功率基础和截断基础的比较。x 轴横跨嵌入维度，y 轴为正余弦基的频率值。

5 结果与讨论

在以下实验中，我们在 RedPajama 数据集的一部分上对 LLaMA-13B 基础模型进行了微调

[5]，并对其进行了修改，使每个数据样本的大小正好为 4096 个 token。我们使用每种位置编码方法进行训练，直到评估损失大致趋于稳定。损失曲线见附录 B。

然后，我们利用 Vicuna 数据集[23]和 LoRA[24]在基础模型上进一步应用了指令微调（IFT）。然而，我们发现，虽然 IFT 确实提高了 LongChat-Lines 的准确率，但它并没有显著改变基础模型能够处理的语境范围（见附录 C）。中的图 5 我们发现这与 WikiQA 变体形成了明显的对比；在 WikiQA 变体中，IFT 是模型产生任何有意义结果的必要条件。因此，对于 LongChat-Lines，我们使用的是非 IFT 模型；而对于 WikiQA，我们则使用了附加 IFT 的更有前途的模型子集进行评估。

5.1 微调上下文长度推断法

LongChat-Lines 我们使用第 4 节中描述的技术对 LongChat-Lines 进行了评估，结果见表 1。鉴于模型是在上下文长度为 4096 的情况下进行微调的，并且在所有情况下都实现了收敛，因此我们预计所有模型都能以不为零的准确率至少执行到 4200。然而，xPos 的情况并非如此，它根本无法执行任务。我们猜测这可能是因为 xPos 与 RoPE 的基础差异太大，模型在微调时无法适应；正如我们在附录 B 中看到的，xPos 的训练和评估损失无法达到与其他方法相同的值。这也可能是在 xPos 实现过程中遇到的数值精度问题造成的。

线性缩放能够成功实现上下文长度外推。值得一提的是，由于基础模型是在上下文长度为 2048 的情况下训练的，因此我们预计因子为 x 的缩放可以实现高达 $2048 - x$ 的非零精度。虽然这是在因子为 4 的线性扩展中观察到的，但我们在表 1 中看到，随着上下文长度的增加（扩展因子为 16），精度下降的速度要快得多。在上下文长度为 17500 时，准确率已经达到 0%，尽管我们天真地认为，这可能是一个合理的预测。

背景 长度	线性 缩放 (系数 =4)	线性 缩放 (因子 =16)	幂基	截断 基数	随机化 位置	xPos
2500	0.7	0.64	0.96	0.42	0.54	0
3600	0.64	0.42	0.64	0.26	0.54	0
4200	0.56	0.56	0.1	0.18	0.3	0
4800	0.66	0.62	0	0.14	0.14	0
7100	0.36	0.4	0	0.04	0.16	0
9400	0	0.22	0	0	0	0
11800	0	0.14	0	0	0	0
14000	0	0.12	0	0	0	0
16000	0	0.1	0	0	0	0
17500	0	0	0	0	0	0
20000	0	0	0	0	0	0
22000	0	0	0	0	0	0

表 1: 在对 LLaMA-13B 进行微调后, 其基本上下文长度为 4096, 本表表示在 LongChat-Lines 上使用不同上下文外推方法进行的评估。精确度为 1.0 表示性能完美, 0.0 表示每次评估都出错。幂基使用的参数为 $k=0.5$ 。截断基使用以下参数: $a=\frac{(1)(2)\pi}{\epsilon}$ 的下界参数 $b=\frac{(2)\pi}{\epsilon}$ 。随机化使用 $a=\frac{(1)(2)\pi}{\epsilon}$ 的下界参数 $b=\frac{(2)\pi}{\epsilon}$ 。所有评估都是在没有额外指令微调的情况下进行的。

在大约 32 000 个上下文长度范围内的性能都很好。我们认为这表明插值方法存在局限性, 并有兴趣在今后的工作中进一步研究。

幂级数基础虽然在最短上下文长度时性能最佳, 但衰减也最快, 在 4200 上下文长度之后根本无法显示任何外推性能。根据表中的结果, 随机位置法似乎是在推断。不过, 这很可能是由于我们评估模型的方式造成的。如第 4。节所述, 在训练时, 模型在 $[\epsilon, 2]$ 范围内均匀采样距离在评估时, 我们并不清楚最佳的位置选择是什么。我们尝试了一系列不同的方法: 大小为 1 的固定距离、 $[\epsilon, 2]$ 中的均匀随机距离和 $[\epsilon, 1]$ 中的均匀随机距离。我们发现后者的外推法效果最好, 因此我们报告了这一结果。我们希望通过进一步降低上限, 可以从模型中得到所需的上下文长度外推法。然而, $[\epsilon, 0.5]$ 及以下的范围大大降低了模型的性能。我们由此得出的结论是, 在不知道其他位置的情况下, 模型无法独立地学习代表每个位置。未来工作的一个有趣方向是在训练 (和评估) 过程中将 Q-proj 和 K-proj 矩阵作为采样位置的条件。

截断基似乎确实能够提供真正的上下文长度外推, 因为它能够在其之前所见过的任何值之外的上下文长度上实现非零精度。虽然其性能会随着长度的增加而下降, 而且目前的表现形式在性能上不如线性缩放, 但我们相信这可能是一个可以带来更好外推性能的研究方向。截断也可以与线性缩放相结合, 我们将在第 5.2 节中讨论。

WikiQA 变体 我们进一步在第 4 节所述的 WikiQA 变体上对线性缩放和截断基方法进行了评估。与检索任务不同的是, 我们发现在没有任何指令微调的情况下, 模型无法成功完成这项任务, 因此我们只对几种感兴趣的方法进行了分析。结果如表 2 和表 3。所示这些结果在很大程度上与 LongChat-Lines 中的模式相吻合--比例因子为 4 的线性缩放能够执行高达 7500 上下文的任务, 但不能超过 7500, 而比例因子为 16 的线性缩放能够超过这一临界值, 但在准确性上会出现一个斜坡。与 LongChat-Lines 的情况一样, 我们发现随着上下文长度的增加, 模型的准确性似乎有所下降。我们再次看到, 截断基能够成功推断出与 LongChat-Lines 相同的上下文长度, 其精确度与比例因子 4 的线性比例相当, 但似乎也无法超过约 8k 的上下文长度。

上下文长度	线性比例（系数=4）	线性缩放 (因子=16)	截断基
2000	0.72	0.69	0.74
3800	0.72	0.73	0.73
7500	0.62	0.70	0.46
15000	0.00	0.68	0.00
24000	0.00	0.56	0.00
32000	0.00	0.18	0.00

表 2：在 AlteredNumeric WikiQA 变体上使用不同上下文分隔方法微调模型的准确率。所有评估均以指令微调方式进行。

上下文长度	线性缩放（因子=4）	线性缩放 (系数=16)	截断基
1900	0.44	0.47	0.46
3800	0.49	0.44	0.55
7600	0.46	0.45	0.36
15000	0.00	0.48	0.00
24000	0.00	0.42	0.00
32000	0.00	0.20	0.00

表 3：使用不同语境外推方法微调的模型在 FreeForm WikiQA 变体上的精度。所有评估均以指令微调方式进行。

5.2 零点线性缩放

在上一节中，我们考察了在微调时使用与评估时相同值的线性缩放的性能。在本节中，我们将研究在评估时使用与模型训练时不同的缩放因子的效果。表 4 显示了这一策略在 LongChat-Lines 上的评估结果。我们发现，一般情况下，如果模型是以 x 的缩放因子训练的，那么模型就能成功地以 $2x$ 的缩放因子评估零镜头（在模型以前能处理的上下文长度范围内，准确率会有所下降）。此外，当比例因子为 16 时，模型似乎无法再通过这种方法增加有效上下文长度。我们还发现，使用 $> 2x$ 进行评估会导致模型崩溃，无法执行任务。

我们的研究表明，在使用截断基础进行微调后，零点线性缩放可以成功应用。有趣的是，对于线性缩放，在评估时使用更长的缩放因子会导致模型之前可以处理的上下文长度的准确性下降，而对于截断基础，情况似乎并非如此--相反，模型可以达到非零准确性的上下文长度范围增加了，而且在模型经过微调的上下文长度上，准确性也提高了。

5.3 比较任务的复杂性

在第 3 节中我们介绍了两项任务，这两项任务特别要求长语境 LLM 从整个文本中提取答案，并认为这些任务对长语境性能的评估可能优于原始易错度。为了分析与这些任务相比，易读性的表现如何，我们报告了 RedPajama 数据集中一个训练有素的模型子集的易读性（见表 5）。当达到模型完全无法处理的上下文长度时（例如，基本 LLaMA 模型超过 2k 或线性标度 4 模型超过 8k），复杂度得分确实会大幅增加。然而，在显示有效范围内长语境能力的下降方面，它们似乎不那么有效。特别是，虽然我们观察到随着上下文长度的增加，表 1 中，但线性标度 4 和截断基础列的 LongChat-Lines 和 WikiQA 变体的性能会出现急剧下降这种下降并没有强烈反映在这些上下文的困惑度得分上。不过，在我们的任务中，线性标度 16 模型的困惑度和准确度似乎是完全相关的。也许最能说明问题的是

上下文长度	Train Scaling= 1 (基本模型)			列车缩放= 4			列车缩放= 16			截断 (缩放= 1)		
	评估 1	评估 2	评估 4	评估 4	评估 8	评估 16	评估 16	评估 32	评估 64	评估 1	评估 2	评估 4
2500	0	0.32	0	0.88	0.64	0	0.64	0.24	0	0.42	0.58	0
3600	0	0.3	0	0.8	0.58	0	0.42	0.26	0	0.26	0.44	0
4200	0	0.18	0	0.86	0.62	0	0.56	0.12	0	0.18	0.28	0
4800	0	0	0	0.86	0.62	0	0.62	0.22	0	0.14	0.26	0
7100	0	0	0	0.64	0.38	0	0.4	0.12	0	0.04	0.04	0
9400	0	0	0	0	0.32	0	0.22	0.12	0	0	0.04	0
11800	0	0	0	0	0.30	0	0.14	0.1	0	0	0.04	0
14000	0	0	0	0	0.1	0	0.12	0.04	0	0	0	0
16000	0	0	0	0	0.12	0	0.1	0.02	0	0	0	0
17500	0	0	0	0	0	0	0	0	0	0	0	0
20000	0	0	0	0	0	0	0	0	0	0	0	0
22000	0	0	0	0	0	0	0	0	0	0	0	0

表 4：使用不同上下文外推方法微调的模型在 LongChat-Lines 上的准确率，然后使用额外的线性比例进行评估。只要评估线性比例大于训练线性比例，就会产生零镜头上下文长度外推。一般来说，在训练长度为 1 和 4 的情况下，评估上下文长度比训练长度增加 2 倍实际上会使可用上下文长度增加一倍，但要付出精度代价（在训练长度为 16 的情况下则不会）。缩放 2 倍后，截断基精度有所提高。如果更激进地放大到列车长度的 4 倍，则会导致模型明显失效。

在模型之间的比较中，截断法的复杂度存在缺陷。根据表 5，截断基在 8k 及以下时表现最佳；然而，在表 1、表 2 和表 3 中，截断基在 8k 上下文时的性能明显低于线性缩放模型。

文献中通常使用易解性来衡量长语境性能[13, 7]，但我们认为，这些结果表明，易解性本身并不足以衡量长语境性能，最好是与其他任务一起使用，以进一步探究 LLM 的能力。

上下文长度	LLaMA 基础	线性缩放 (系数=4)	线性缩放 (因子=16)	截断基数
512	4.06	4.06	4.05	3.79
1k	3.88	3.87	3.86	3.63
2k	3.79	3.75	3.74	3.52
4k	9022	3.66	3.66	3.46
8k	7198	3.79	3.97	3.78
16k	5141	14902	5.43	15793
24k	4980	21236	8.73	13929
32k	4408	55480	98.12	12534

表 5：不同模型在不同上下文长度下对 RedPajama 数据集的保留评估集的复杂度得分。评估长度为 256 个词组，给模型的提示是文档的前 N - 256 个词组，其中 N 是我们评估的上下文长度。当上下文长度过长，模型无法有效处理时，复杂度就会急剧上升；然而，在模型可以处理的范围内，复杂度似乎对上下文使用退化的敏感度低于 LongChat-Lines 任务，并且不会像 LongChat-Lines 或 WikiQA 那样出现模型之间的排名。

5.4 分析问答定位

对于 WikiQA 变体，我们对答案和问题的位置影响进行了分层分析。如第 4 节所述我们考察了将答案放在文档前 10%、后 10% 或其他随机位置的影响。我们还考察了将问题放在提示开头或结尾的影响。结果如表 6 和表 7 这是所示，在系数为 16 的线性缩放模型上进行的，并进行了额外的指令微调。

上下文长度	答案位置			问题位置	
	开始	中间	结束	起点	结束
2000	0.74	0.68	0.66	0.70	0.69
3800	0.70	0.67	0.66	0.63	0.73
7500	0.69	0.63	0.65	0.61	0.70
15000	0.68	0.68	0.69	0.68	0.68
24000	0.30	0.26	0.50	0.14	0.56
32000	0.13	0.13	0.23	0.15	0.18

表 6: AlteredNumericQA 任务的分层准确率分析。对于答案, "开始"指文档的前 10%, "结束"指文档的最后 10%, "中间"指任何其他位置。对于问题, "开始"指的是将问题放在语言提示的其余部分之前, "结束"指的是将问题放在最后。报告结果基于 LLaMA-13B 微调, 线性量表为 16, 并应用了 IFT。

语境长度	答案位置			问题位置	
	开始	中间	结束	起点	终点
1900	0.37	0.40	0.36	0.27	0.47
3800	0.40	0.30	0.32	0.24	0.44
7600	0.35	0.34	0.35	0.24	0.45
15000	0.44	0.43	0.45	0.40	0.48
24000	0.18	0.20	0.40	0.10	0.42
32000	0.07	0.11	0.18	0.04	0.20

表 7: FreeFormQA 任务的分层准确率分析。对于答案, "开始"指文档的前 10%, "结束"指文档的最后 10%, "中间"指任何其他位置。对于问题, "开始"是指将问题放在语言提示的其余部分之前, "结束"是指将问题放在最后。报告结果基于 LLaMA-13B 微调, 线性量表为 16, 并应用了 IFT。

我们的目标是以 [22] 的。类似分析为基础然而, 我们无法在 LongChat-13B (16K) 模型 (我们的建模方法与之最为相似) 上复制该论文中的结果。在 FreeFormQA 和 AlteredNumericQA 中, 我们没有观察到答案在提示语中的位置以及模型在 15k 上下文长度内的准确性有明显的变化趋势。对于 AlteredNumericQA 来说, 问题的位置似乎也没有明显的影响, 但对于 FreeFormQA 来说, 则有明显的影响, 因为将问题放在最后似乎会显著提高准确率。不过, 在 24k 和 32k 上下文长度下, 我们在两个数据集中都看到了明显的迹象, 即答案在末尾和问题在末尾的准确率都比放在其他位置的准确率高。这些结果与 [22] 中的结果形成了鲜明的对比。我们从中得到的启示是, 在 LLM 的性能中, 关于它们如何很好地利用上下文的所有部分, 可能存在大量的任务条件变异; 即使是任务构建中的微小差异, 也会导致观察到的趋势出现巨大差异。

6 结论与局限性

在本文中, 我们研究了多种方法来微调预训练的基础 LLaMA 和 LLaMA2 LLM, 这些方法具有有限的上下文长度, 使其能够将零点推断到新的、更长的上下文长度。我们使用plexxity以及两个探究长语境性能的自定义任务对这些方法进行了比较; 我们发现, 与plexxity相比, 自定义任务能提供对长语境性能更精细的理解。我们发现, 线性插值法在上下文长度外推法中表现最佳, 并发现使用我们称之为 *截断基* 的新基的潜力。我们发布了三个模型, 我们称之为 "长颈鹿" (Giraffe), 它们使用线性内插法扩展了基础 LLaMA 和 LLaMA 2 模型的上下文长度。

本文所介绍的工作还有很大的发展空间。我们注意到, 随着上下文长度的增加

随着上下文长度的增加，所有方法在我们的评估任务中都会出现准确性下降的情况，尽管困惑度通常保持合理，模型仍能产生一致的输出结果。这是我们有兴趣解决的一个缺陷，而且我们认为，这对于声称模型具有 "真正的" 长语境外推能力是必要的。

这项工作的局限性在于，我们只在单一文档数据集上进行了易错性分析。今后的工作可以考虑在其他数据集上重复这一分析。此外，我们还特别关注了从预先训练的基础模型，尤其是使用 RoPE 位置编码训练的 LLaMA 和 LLaMA 2 模型进行上下文长度外推的情况。未来的工作可以研究本文的分析是否可以扩展到其他位置编码类型和模型。未来的工作还可以解决线性插值方法的局限性。我们在 "LongChat-Lines" 任务中看到了一些证据，特别是随着比例因子的增加，准确度会下降。这种方法的比例系数的极限是多少？是否有一个点，超过这个点，它就无法改善它能处理的上下文范围？此外，截断基方法似乎显示出了真正的外推能力，那么这种方法是否可以进行修改，以获得与线性插值法相等或超越线性插值法的能力呢？我们相信，这些都是未来值得关注的潜在方向。

参考文献

- [1] Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N. Gomez、Lukasz Kaiser 和 Illia Polosukhin。关注就是一切，2023。
- [2] 丹·亨德里克斯（Dan Hendrycks）、科林·伯恩思（Collin Burns）、史蒂文·巴萨特（Steven Basart）、安迪·邹（Andy Zou）、曼塔斯·马泽卡（Mantas Mazeika）、宋黎明（Dawn Song）和雅各布·斯坦哈特（Jacob Steinhardt）。测量大规模多任务语言理解能力，2021年。
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, 穆罕默德·巴伐利亚、克莱门斯-温特、菲利普·蒂莱、费利佩·佩特罗斯基-苏奇、戴夫·卡明斯、马蒂亚斯-普拉珀特、福蒂奥斯-昌特齐斯、伊丽莎白-巴恩斯、阿里尔-赫伯特-沃斯、威廉·赫伯根-古斯、亚历克斯-尼科尔、亚历克斯-帕诺、尼古拉斯-特扎克、唐杰、伊戈尔-巴布什金、苏奇尔-巴拉吉、尚塔努-詹恩、威廉-桑德斯、克里斯托弗-赫塞、安德鲁-N.Carr、Jan Leike、Josh Achiam、Vedant Misra、Evan Morikawa、Alec Radford、Matthew Knight、Miles Brundage、Mira Murati、Katie Mayer、Peter Welinder、Bob McGrew、Dario Amodei、Sam McCandlish、Ilya Sutskever 和 Wojciech Zaremba。《评估基于代码训练的大型语言模型》，2021 年。
- [4] Leo Gao、Stella Biderman、Sid Black、Laurence Golding、Travis Hoppe、Charles Foster、Jason Phang、Horace He、Anish Thite、Noa Nabeshima、Shawn Presser 和 Connor Leahy。这堆数据用于语言建模的 800GB 多样文本数据集，2020 年。
- [5] Together Computer.Redpajama：重现骆驼训练数据集的开源配方，2023 年 4 月。
- [6] Ofir Press、Noah A. Smith 和 Mike Lewis。训练短，测试长：具有线性偏差的注意力可实现输入长度外推，2022年。
- [7] 孙玉涛、董力、巴伦·帕特拉、马书明、黄绍翰、阿隆·本海姆、维什拉夫-乔杜里、宋霞和魏福如。长度可外推变压器，2022 年。
- [8] OpenAI.Gpt-4技术报告，2023年。
- [9] 人类学。介绍克劳德，2023年。
- [10] Hugo Touvron、Thibaut Lavril、Gautier Izacard、Xavier Martinet、Marie-Anne Lachaux、Timothée Lacroix、Baptiste

Rozier, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand

Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

- [11] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov 和 Thomas Scialom. 喇嘛 2：开放式基础和微调聊天模型，2023 年。
- [12] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, Kenton Lee, Kristina N. Toutanova, Llion Jones, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 自然问题：问题解答研究的基准。《计算语言学协会论文集》，2019。
- [13] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer：带旋转位置嵌入的增强变换器》，2022 年。
- [14] 彼得-肖、雅各布-乌斯科雷特和阿希什-瓦斯瓦尼。利用相对位置表示的自我注意，2018 年。
- [15] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 用 mt-bench 和 chatbot arena 评判 llm-as-a-judge，2023。
- [16] 我在训练 superhot 时学到的东西》，2023 年。
- [17] 陈寿元、Sherman Wong、陈良健、田远东。通过位置插值扩展大型语言模型的上下文窗口，2023 年。
- [18] Keyulu Xu, Mozhi Zhang, Jingling Li, Simon S. Du, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 神经网络如何外推：从前馈到图神经网络，2021 年。
- [19] Anian Ruoss, Grégoire Delétang, Tim Genewein, Jordi Grau-Moya, Robert Csordás, Mehdi Bennani, Shane Legg, and Joel Veness. 随机位置编码提升反式编码器的长度泛化，2023。
- [20] anadim. 大型语言模型的小检索测试，2023 年 5 月。
- [21] 李大成*、邵如林*、谢安泽、盛颖、郑连民、约瑟夫-E-冈萨雷斯、扬-斯托伊卡、马学哲、张浩。开源 llms 真正能承诺的上下文长度有多长？
- [22] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 迷失在中间：语言模型如何使用长语境》，2023 年。
- [23] 郑连民、蒋伟林、盛莹、庄思源、吴章浩、庄永浩、林紫、李卓涵、李大成、Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 用 mt-bench 和聊天机器人竞技场评判 LLM-as-a-judge》，2023 年。

[24] 胡爱华、沈业龙、菲利普·沃利斯、朱泽元、李远志、王申、王璐和陈伟柱。Lora：大型语言模型的低秩适配，2021年。

上下文长度	LLaMA 2 线性 (8)
2500	0.48
3600	0.42
4200	0.32
4800	0.74
7100	0.56
9400	0.50
11800	0.50
14000	0.42
16000	0.38
17500	0.14
20000	0.14
22000	0.08
26000	0.00
30000	0.00

表 8: LLaMA 2 在 LongChat-Lines 上的性能，比例因子为 8。

上下文 长度	LLaMA 2 线性 (8)	
	AltQA	FFQA
2k	0.72	0.56
4k	0.76	0.55
8k	0.71	0.56
16k	0.59	0.44
24k	0.36	0.28
32k	0.15	0.10

表 9: LLaMA 2 在比例因子为 8 的 WikiQA 变体上的性能。

A LLaMA 2

在我们完成本文时，Meta 发布了 LLaMA 2 [11]。我们验证了 LLaMA 2 可以通过线性插值法实现类似的上下文长度外推结果。我们采用了与第 4 节所述相同的方法在经过修改的 RedPajama 数据集的一部分上训练 LLaMA 2-13B，使每个数据样本的大小正好为 4096 个标记。然后，我们还对 Vicuna 数据集进行了指令微调。表 8 和表 9 显示了测试结果。

我们可以看到，该模型在长达 22000 上下文长度的 LongChat-Lines 上的准确率不为零，超过了我们在主要论文中测试的任何模型。该模型还能在 WikiQA 变体上实现非零性能，最高可达 32k 上下文。不过，随着上下文长度的增加，我们确实发现这两项任务的准确性都在下降。同样值得注意的是，这两个任务的准确度都略低于 LLaMA 1，在上下文长度为 16 的情况下，这两个模型都能取得非零的结果。

B 损失曲线

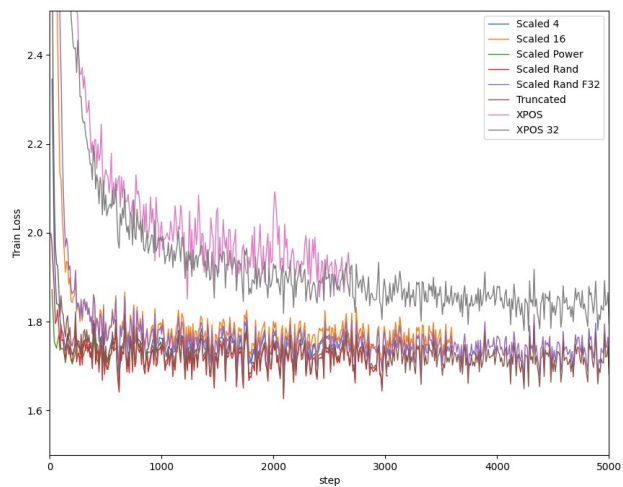


图 3：从 RedPajama 数据集中提取的 4096 个 token 样本的初始拟合运行期间模型的训练损失曲线。

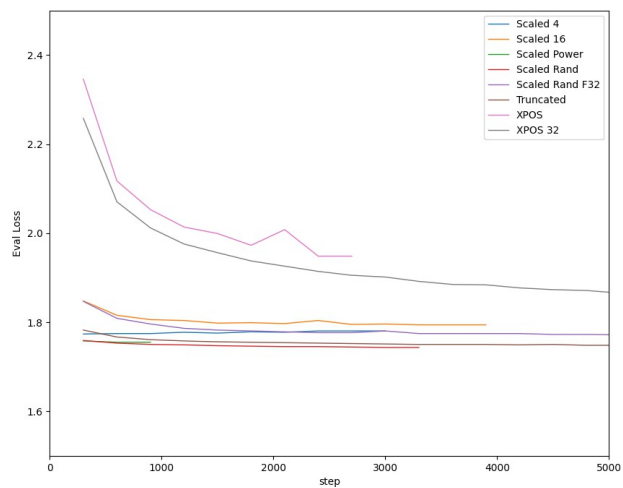


图 4：在从 RedPajama 数据集提取的 4096 个标记样本上进行初始拟合运行期间模型的评估损失曲线。

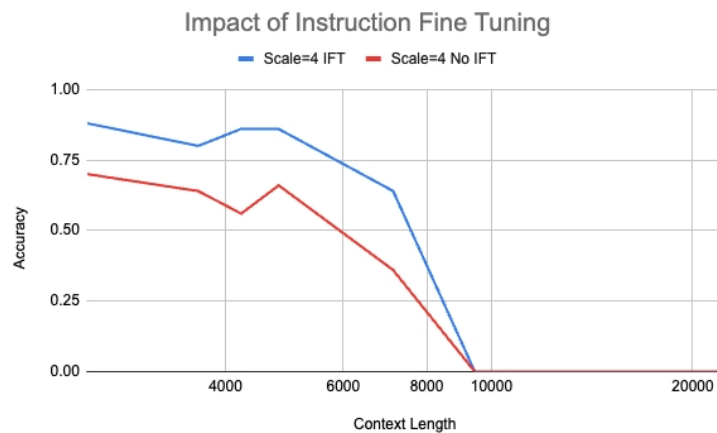


图 5：在 LongChat-Lines 上进行的 IFT 与非 IFT 的比较（线性缩放为 4）。虽然 IFT 提高了准确率，但并没有扩大模型获得非零准确率的语境范围。

C IFT 的影响

正如正文第 5 节所述，我们发现使用 Vicuna 数据集进行指令微调确实提高了 LongChat-Lines 的准确率，但并没有改变特定模型的非零语境范围。图 5 显示的是规模为 4 的线性插值模型。